CAPSTONE PROJECT | FINAL CONSOLIDATED REPORT | SPRINTS 1-5

# Container Image Vulnerability Scanner with Reporting

A policy-driven container scanning system with auditable reports, CI gates, Slack notifications, governed exceptions and persistent vulnerability monitoring.

**Submitted by:** Tabrez Ajaz

**GitHub account:** TabrezAjaz

**Report date:** 30 October 2026

**Implementation branch:** main

**Scanner engine:** Trivy 0.74.0

**Python wrapper:** 0.5.0

**Implementation baseline:** 399d1af

**Integrated deterministic suite:** 77 tests

Public source repository | Operating documentation | Printable PDF edition

> **Delivery statement:** Sprint 1-5 implementation is merged into main. Jenkins Slack transport has been verified. GitHub Slack secret entry, rotation of the previously exposed webhook and channel-side visual evidence remain account-owner actions; they are not represented as completed.

This report consolidates implementation, original artifacts, genuine browser captures and verified execution records. Milestone reports remain historical appendices. Live vulnerability counts depend on the scan date, image and advisory database.

# Contents


1. Executive Summary
2. Objectives and Traceability
3. Implementation and Repository
4. Deployment Architecture
5. Sprint 1: Setup and Scanning
6. Scanner and Gate Contract
7. Sprint 2: CI/CD Integration
8. Sprint 3: Reports and Slack
9. Scheduled Automation
10. Sprint 4: History and Monitoring
11. Sprint 5: Exceptions and Reliability
12. Verification Matrix
13. Security and Limitations
14. Reproduction and Handover
15. Challenges and Decisions
16. Evidence Register
17. Milestone Appendices
18. Conclusion


# 1. Executive Summary

The project converts Trivy scan results into a consistent security control rather than an isolated command-line scan. The Python wrapper loads configuration, invokes a pinned scanner, validates the result structure, applies approved exceptions, writes normalized evidence and returns a stable policy or operational exit code. GitHub Actions and Jenkins use the same implementation.

A strict reporting layer renders JSON, HTML and PDF from one canonical model and publishes SHA-256 manifests. Slack messages distinguish severity and scheduled portfolio summaries. A separate local monitoring deployment imports original observations into SQLite and exposes a read-only Flask/Chart.js dashboard and Prometheus metrics; Grafana queries Prometheus for sampled time-series views.

The final implementation includes reasoned, scoped and expiring exceptions, preserved suppression audit, fresh rescan lineage, classified retries, process deadlines and atomic artifact writes. The integrated suite contains 77 deterministic tests. GitHub main run #31, for baseline `399d1af`, completed successfully. Its Slack step was skipped, not delivered, because the repository secret is absent.

# 2. Objectives and Requirement Traceability

| Sprint | Acceptance objective | Delivered implementation | Primary evidence |
|---|---|---|---|
| 1 | Establish repository, install scanner and perform image scans | Python wrapper, YAML configuration, pinned Trivy and normalized findings | Fresh core verification log; original baseline/rescan artifacts; latest real CI scan |

| Sprint | Acceptance objective | Delivered implementation | Primary evidence |
|---|---|---|---|
| 2 | Make vulnerability scanning a CI/CD gate | Shared severity policy, stable exits, GitHub/Jenkins pipelines and retained evidence | Real passing and intentionally blocked GitHub/Jenkins captures |
| 3 | Generate reports, notify and automate summaries | JSON/HTML/PDF, checksum manifest, Slack transport, daily/weekly jobs | Generated report bundle; main HIGH/CRITICAL delivery records; default-cadence runs |
| 4 | Track findings/history with filters and monitoring | SQLite, Flask API/UI, Prometheus exporter and provisioned Grafana | Desktop/mobile captures, target UP, Grafana panels and browser-check JSON |
| 5 | Manage accepted risk, rescans and failures | Expiring scoped policy, suppression audit, fresh lineage, bounded retries and strict validation | Approved/expired artifacts, rescan lineage, audit report and 77-test JUnit evidence |

**Scope:** OS-package vulnerabilities are scanned by default, including unfixed findings. Language dependency scanning, Teams/email delivery, automatic registry-digest watches and a publicly authenticated monitoring service are outside the delivered scope. The submission covers Sprints 1-5, not a separately claimed Sprint 6.

# 3. Implementation and Repository

| Component | Technology / location | Responsibility |
|---|---|---|
| Scanning and policy | Trivy 0.74.0; scanner/trivy_scanner.py; scanner/exceptions.py | Execute scans, normalize findings, apply exceptions and enforce exit contracts |
| Configuration | config.yaml; exceptions/allowlist.yaml; .env.example | Non-secret policy defaults and documented environment overrides |
| Reporting | reporters/; Jinja2; ReportLab | Strict artifact loading, canonical model, JSON/HTML/PDF and integrity manifests |
| Notification | notifiers/ | Validated Slack webhook transport with bounded retries and mention sanitization |
| History and metrics | dashboard/; Flask; SQLite; prometheus-client | Idempotent imports, read-only API, exact finding transitions and /metrics |
| Visualization | Vendored Chart.js and Lucide; Prometheus 3.2.1; Grafana 11.6.0 | Local scan exploration and sampled monitoring dashboards |
| CI and tests | .github/workflows/; ci/; tests/ | Real scans, gate/report orchestration, deterministic tests and evidence archives |

CI uses Python 3.12; local verification has used Python 3.14. Python dependencies have bounded version ranges, not a claim of fully locked transitive dependencies. Runtime databases, raw operational outputs, credentials and virtual environments are excluded from Git. Small public demonstration artifacts are intentionally archived under docs/evidence.

Milestone branches preserve sprint snapshots while main contains the complete delivered implementation. An authorized email-only history correction changed commit IDs without changing any of the 32 commit trees. Historical CI URLs and screenshot hashes still describe their original executions; they were not relabelled as new runs.

# 4. Implemented Deployment Architecture

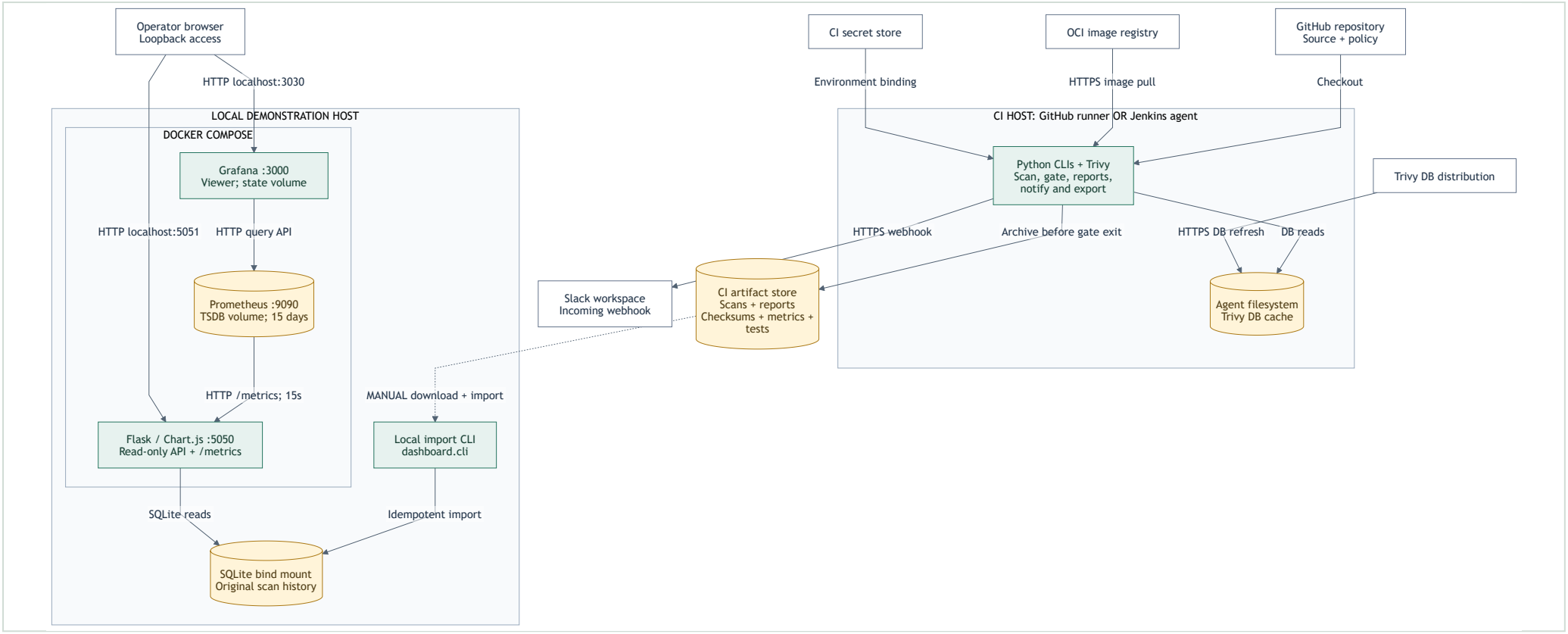



Figure 1. Deployment architecture, not a sprint roadmap. The local monitoring host is separate from hosted CI. Dotted artifact transfer is a manual download/import, not an implemented automatic push.

Full-size vector | High-resolution PNG | Editable Mermaid source

GitHub runners and Jenkins agents are alternative execution environments. The scanner remains a short-lived CLI. The local persistent Flask exporter serves SQLite observations after scans finish; Prometheus scrapes it every 15 seconds and Grafana queries Prometheus. SQLite keeps original scan timestamps, while Prometheus stores samples only after scraping starts. The old image is a *Historical Sprint 1 Component and Workflow Overview*, not this final deployment architecture.

# 5. Sprint 1: Foundation and Basic Scanning

The initial structure separated scanner, configuration, CI, reporting, notification, exceptions, tests and documentation. Trivy was chosen as the underlying scanning engine rather than recreating advisory matching in Python. Registry-reference scans do not require mounting the Docker socket. The scanner downloads/refreshes its advisory database and uses the local cache during analysis.

Current core acceptance is reverified by an original command-output log, fresh normalized clean-image and vulnerable-image artifacts, and the latest main CI scan. These are final-version checks, not backdated Sprint 1 captures.

| Fresh final-version scan | Original scan ID and UTC time | Observed outcome |
|---|---|---|
| hello-world:latest | `27c31b93-4f66-423d-8dd4-6fbae32a70da` 2026-10-29T20:54:15.297881Z | 0 findings; 0 blocking; exit 0 |
| alpine:3.19 | `59b455b0-e86c-40a7-8bbe-0ff1ef0b9a21` 2026-10-29T20:54:17.970747Z | 10 active findings including 2 HIGH; 2 blocking; exit 1 |

The original log records the pinned engine and cached database timestamps. Raw artifact timestamps are UTC; the local Python log clock uses the host timezone. These scans are evidence of the final wrapper version, not a reconstruction of the first development session.

> **Evidence provenance:** several older Sprint 1 terminal-style figures were rendered from a static HTML evidence page. They are historical presentation panels, not independent terminal screenshots, and are deliberately not reused as primary execution proof in this final report. The same rule excludes small reconstructed Sprint 3 panels.

Original dated observations remain available in baseline.json and rescan.json. Their source fields, UUIDs, timestamps, scanner versions and full findings provide inspectable evidence beyond a screenshot.

# 6. Scanner and Policy Contract

```
CLI / environment / YAML configuration
  -> validate policy and exact exception scopes
  -> invoke Trivy and validate result structure
  -> normalize raw findings
  -> separate approved, unexpired findings into suppression audit
  -> recompute active severity counts and gate
  -> atomically write normalized scan evidence
  -> return pass / policy / configuration / operational exit
```

| Exit | Meaning | Expected action |
|---|---|---|
| 0 | Completed and passed the configured policy | Continue pipeline |
| 1 | Completed but active findings violate the severity gate | Intentional build block; retain reports and artifacts |
| 2 | Invalid CLI/configuration/exception policy | Correct the input; never claim the image is clean |
| 3 | Scanner/registry/timeout/parsing/write operation failed | Investigate operational failure; fail closed |

Precedence is CLI > environment > YAML > defaults. The default threshold is HIGH, with CRITICAL also blocking. Input validation, explicit timeout handling and malformed-output rejection prevent an error or empty malformed object from appearing as a safe scan. The reference environment file is documentation, not automatically loaded dotenv configuration.

# 7. Sprint 2: CI/CD Integration

GitHub Actions and Jenkins install the pinned engine, refresh the database, run tests and invoke the same wrapper. The scan result is retained while valid artifacts are rendered and archived. Final gate enforcement restores an intentional policy block even when reports and notifications succeed. An upload attempt does not guarantee evidence exists after every possible infrastructure failure.

| Historical verification | Observed outcome | Interpretation |
|---|---|---|
| GitHub gate run #6 | Success with evidence artifact | Passing control path |
| GitHub gate run #5 | Exit 1 with evidence artifact | Successful policy enforcement, not a broken implementation |
| Jenkins Sprint 2 build #2 | hello-world:latest passed; 16 tests at that milestone; JSON archived | Clean image path |
| Jenkins Sprint 2 build #3 | alpine:3.19 blocked with 2 HIGH findings; JSON archived | Threshold block preserved evidence |
| Latest baseline main run #31 | Success for 399d1af: tests, real scan, reports and upload | Current integrated implementation; Slack skipped |

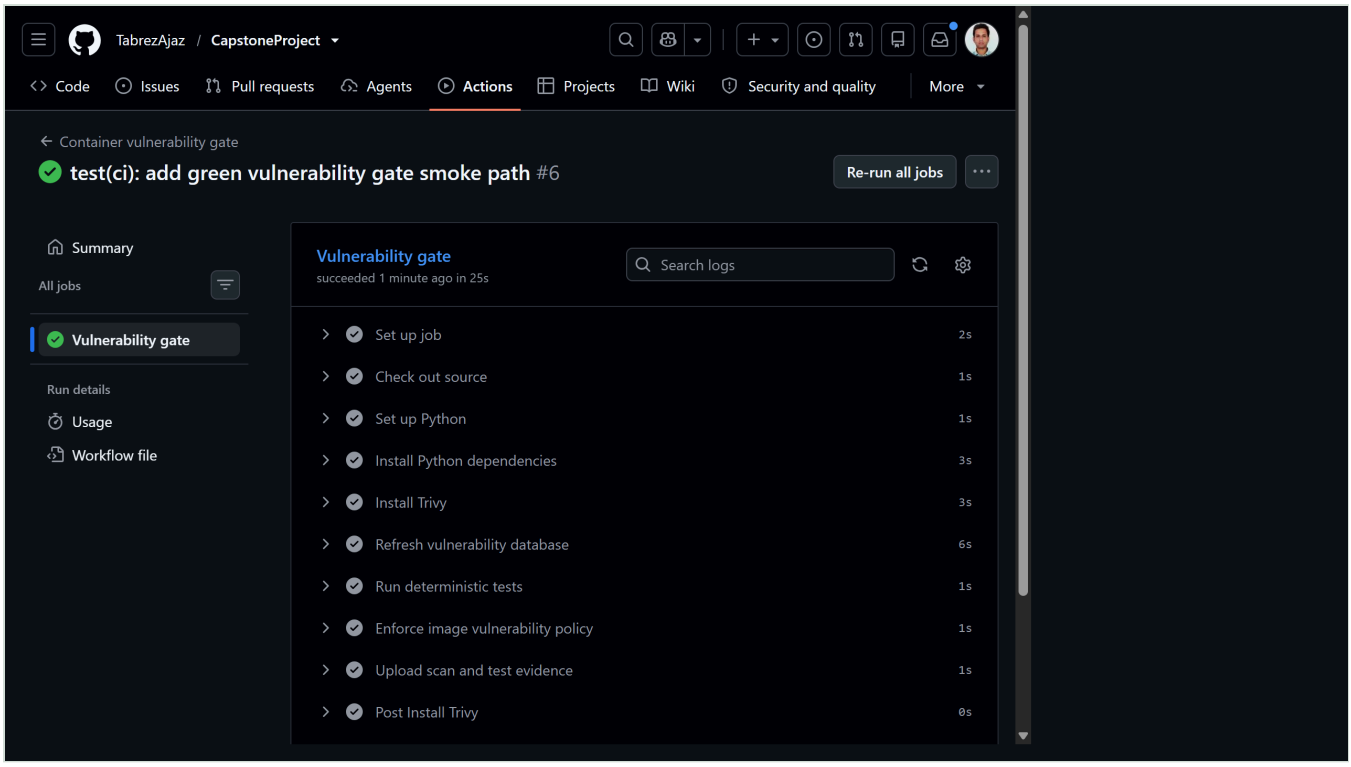



Figure 2. Genuine GitHub browser capture of the passing milestone pipeline. Historical test counts and dates are not presented as current totals.

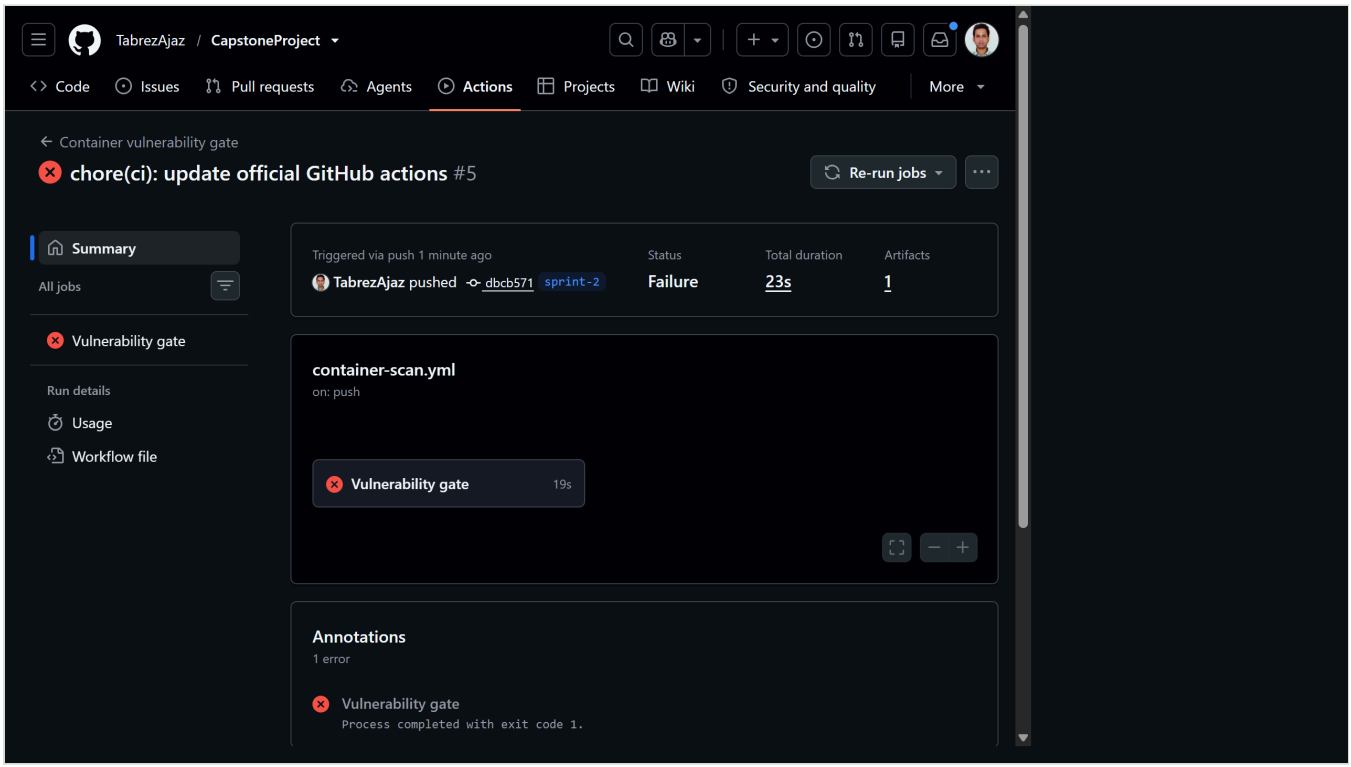



Figure 3. Genuine blocked GitHub run: the red outcome demonstrates the security gate.

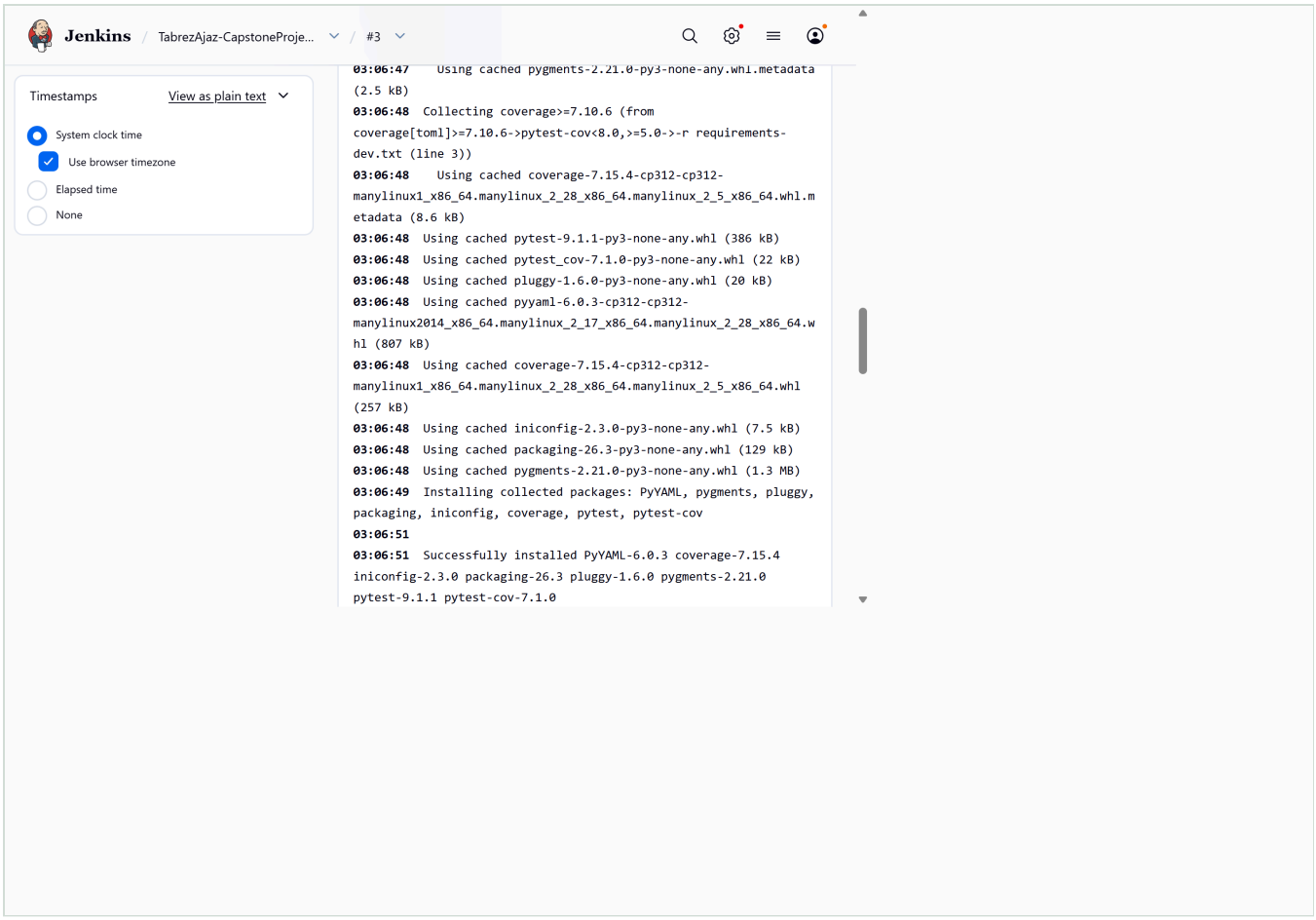



Figure 4. Genuine Jenkins console capture for the intentional HIGH-threshold block.

Machine-readable current verification: run metadata, step outcomes and uploaded artifact metadata. The public run URL is 40951355890 / #31.

# 8. Sprint 3: Reports and Slack Notifications

Schema 1.0 normalized scans feed a strict loader that validates severity counts, policy state and suppression audit. One canonical model generates JSON, autoescaped HTML and PDF. A manifest records output filename, media type, byte size and SHA-256. Checksums demonstrate integrity against a trusted manifest; they do not prove authorship or approval identity.

The archived approved-scan report is available as JSON, HTML and PDF, with its original checksum manifest. All outputs retain the same report identity.

Slack is the implemented provider. Webhooks are injected from environment/CI credentials, validated as permitted HTTPS Slack endpoints and never embedded in project configuration. Messages neutralize mass mentions and limit scan-controlled text. Delivery retries transient HTTP/network conditions and fails operationally after exhaustion.

| Jenkins Sprint 3 execution | Evidence-based result |
| --- | --- |
| Main build #4: HIGH alert | Intentional policy failure; Slack delivered in one attempt; tests/reports retained |

| Jenkins Sprint 3 execution | Evidence-based result |
|---|---|
| Main build #5: CRITICAL alert | Intentional policy failure; Slack delivered in one attempt; reports retained |
| Daily #16 / Weekly #4 | SUCCESS; real Slack transport delivery and archived summary evidence |
| Daily #17 / Weekly #5 | SUCCESS with saved defaults, correct cadence and delivery in one attempt |

These are verified delivery logs, not screenshots proving channel-side receipt. Original controller links: HIGH #4, CRITICAL #5, Daily #17, Weekly #5. Academic Jenkins may require login and is subject to retention.

Selected original console lines already recorded in the Sprint 3 milestone report are reproduced below. Execution names are contextual labels; these are not newly captured console sessions.

| Execution | Original delivery line |
|---|---|
| HIGH main #4 | `[2026-10-29T06:37:57.366Z] Slack notification delivered in 1 attempt(s)` |
| CRITICAL main #5 | `[2026-10-29T06:38:56.071Z] Slack notification delivered in 1 attempt(s)` |
| Daily #16 | `[2026-10-29T06:39:35.504Z] Slack notification delivered in 1 attempt(s)` |
| Weekly #4 | `[2026-10-29T06:40:17.337Z] Slack notification delivered in 1 attempt(s)` |
| Daily #17: saved defaults | `[2026-10-29T18:28:40.484Z] Slack notification delivered in 1 attempt(s)` |
| Weekly #5: saved defaults | `[2026-10-29T18:27:53.087Z] Slack notification delivered in 1 attempt(s)` |

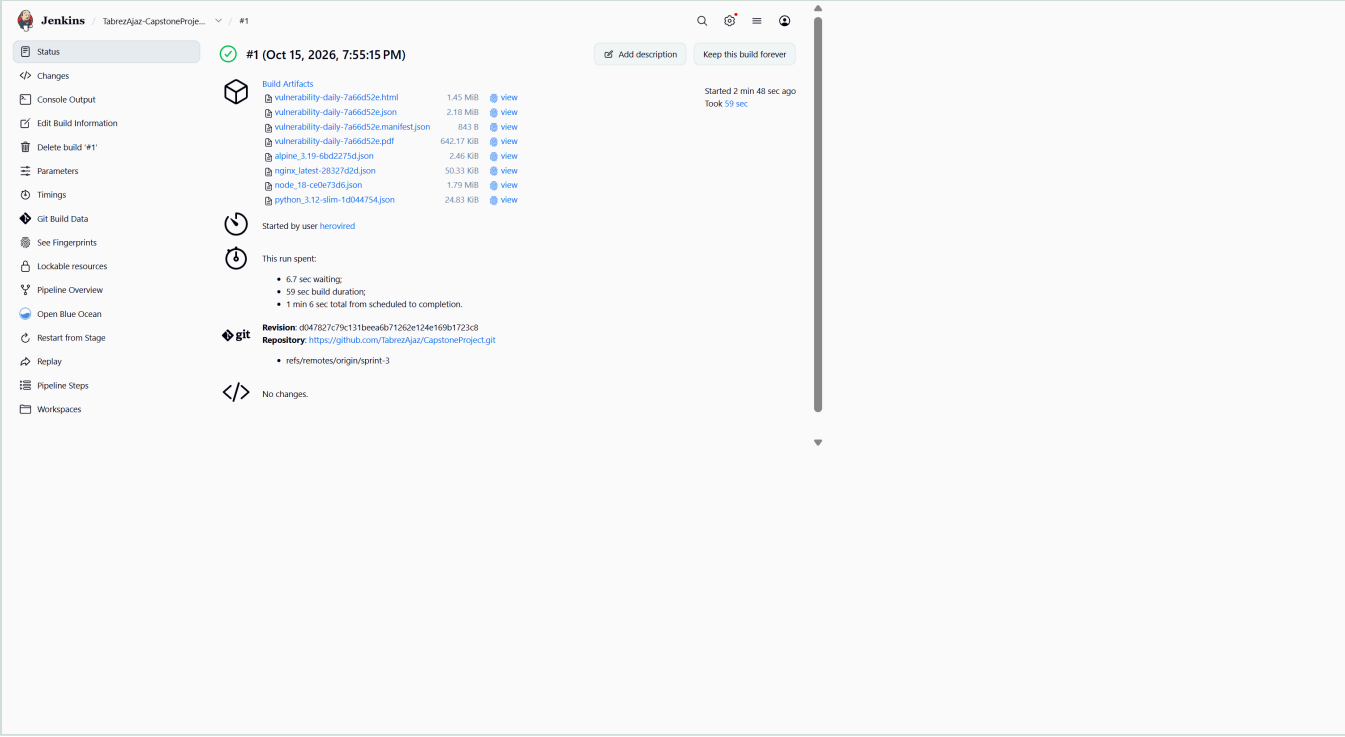



Figure 5. Genuine milestone daily-summary browser capture. Later default-parameter delivery runs are identified separately above; this image is not relabelled as build #17.

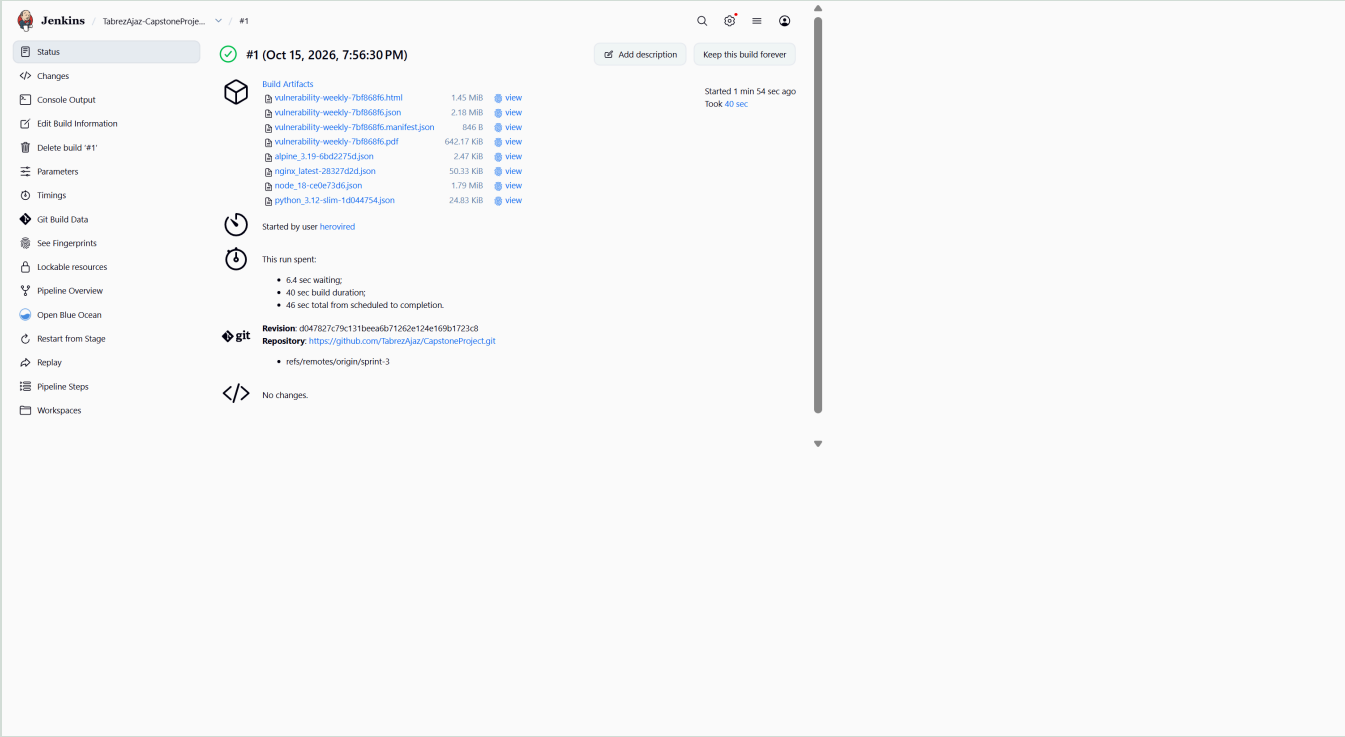



Figure 6. Genuine milestone weekly-summary browser capture, not a screenshot of a Slack channel.

# 9. Scheduled Automation

| Platform | Configured cadence | Validation boundary |
|---|---|---|
| GitHub Actions | Daily 02:23 UTC; Monday 03:41 UTC | Workflow present on main. Manual daily summary run #1 succeeded; its Slack step was skipped. |
| Jenkins daily job | H 2 * * * UTC | Saved trigger and default auto cadence/NOTIFY=true verified; manually triggered default build #17 succeeded. |
| Jenkins weekly job | H 3 * * 1 UTC | Job name ending -Weekly selects weekly in auto mode; manually triggered default build #5 succeeded. |

Scheduled summaries refresh the database and rescan configured images. A policy block is a valid portfolio summary result; operational failures fail the summary job. A successful manual dispatch verifies the workflow and defaults, not that a cron event actually fired. GitHub schedules also depend on platform scheduling/availability.

# 10. Sprint 4: Persistent History and Monitoring

The local importer accepts normalized scan artifacts, not report JSON or raw Trivy output. SQLite imports are transactional and idempotent by scan ID; conflicting ID reuse is rejected. Flask exposes a read-only API, history filters, scan details, approved-exception audit, JSON download and Prometheus exposition. Imports remain a local CLI action.

| Service | Loopback URL | Persistence / role |
| --- | --- | --- |
| Optional direct local Flask server | 127.0.0.1:5050 | Local development demonstration using the same history |
| Container Flask exporter/UI | 127.0.0.1:5051 | Read-only HTTP surface; host SQLite bind mount |
| Prometheus | 127.0.0.1:9095 | 15-second scrapes; 15-day TSDB retention; named volume |
| Grafana | 127.0.0.1:3030/d/container-security | Provisioned datasource/eight panels; anonymous Viewer; named volume |

Latest active findings are pending. New and resolved findings compare exact advisory/package pairs against the previous stored observation. Suppressed pairs remain in this comparison, so accepting risk is not falsely reported as remediation. First observations are baselines. These transition metrics are gauges, not fabricated cumulative counters.

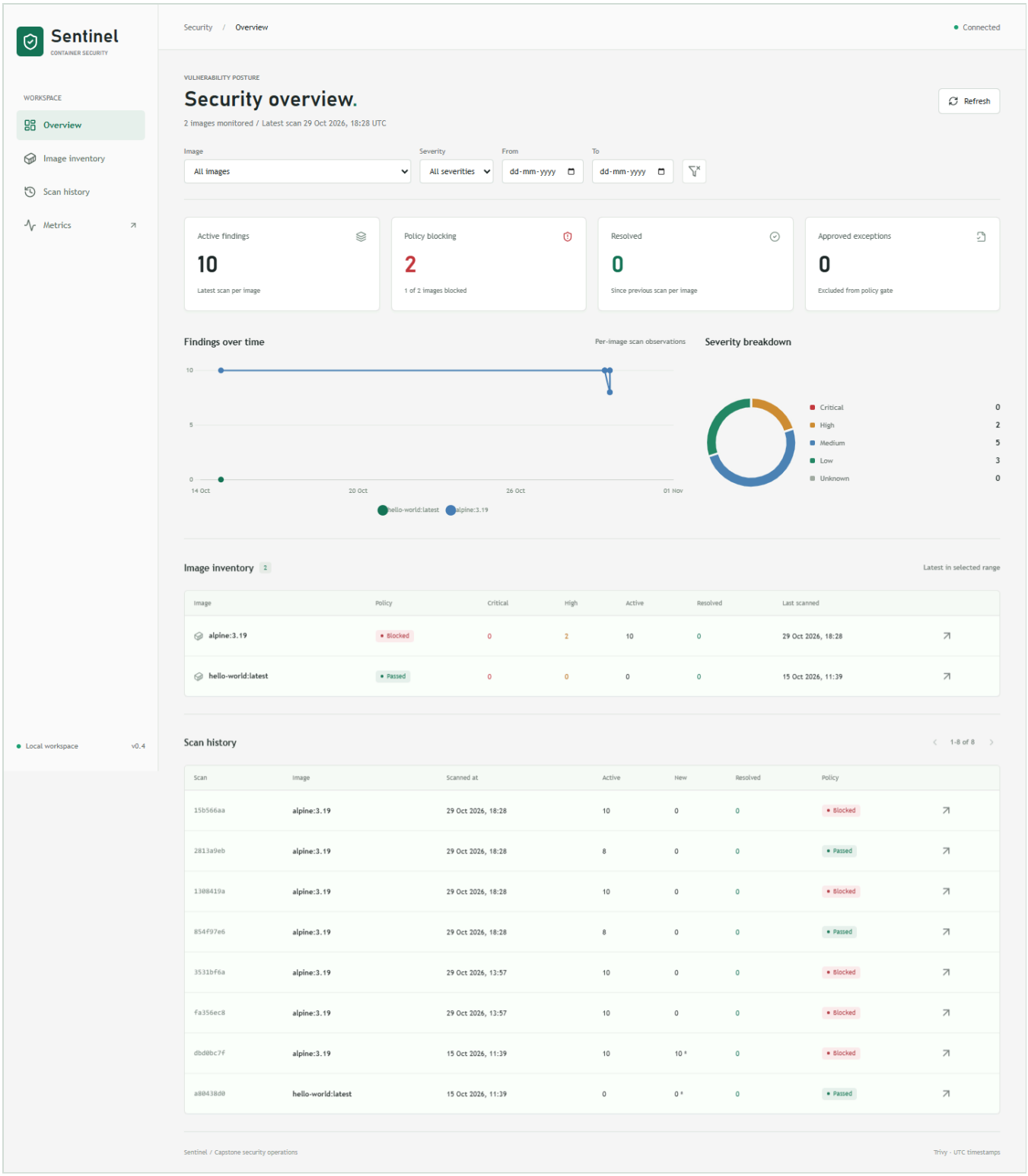



Figure 7. Genuine desktop capture. Displayed counts reflect the stored observations at capture time, not invented sample data.

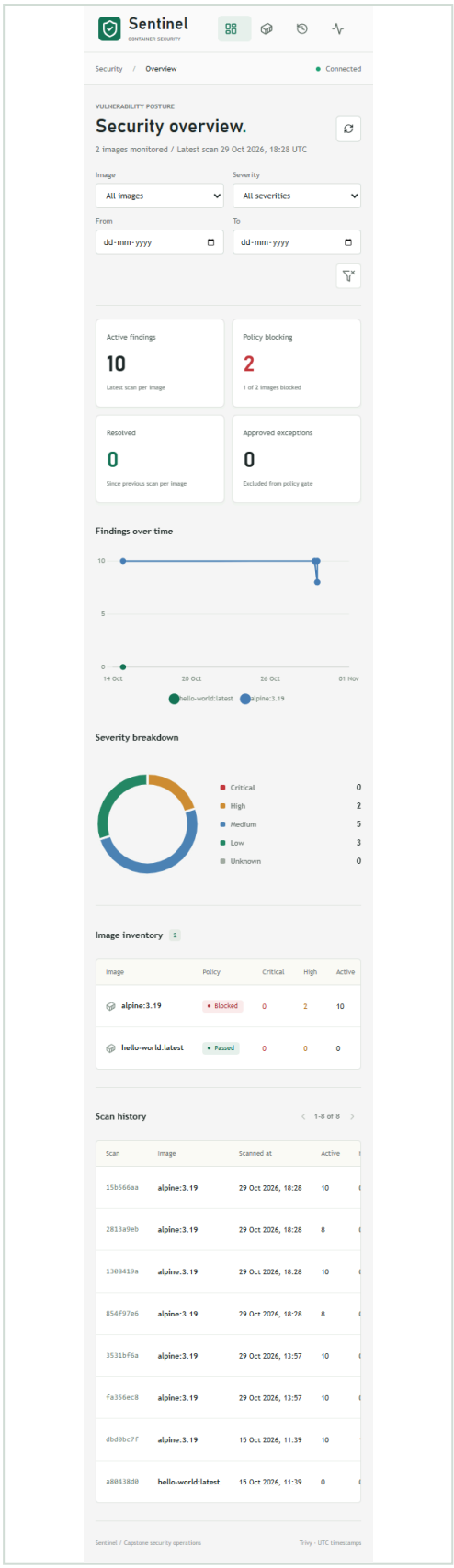



Figure 8. Genuine mobile capture; exact 390x844 checks confirmed readable controls, nonblank charts and no horizontal page overflow.

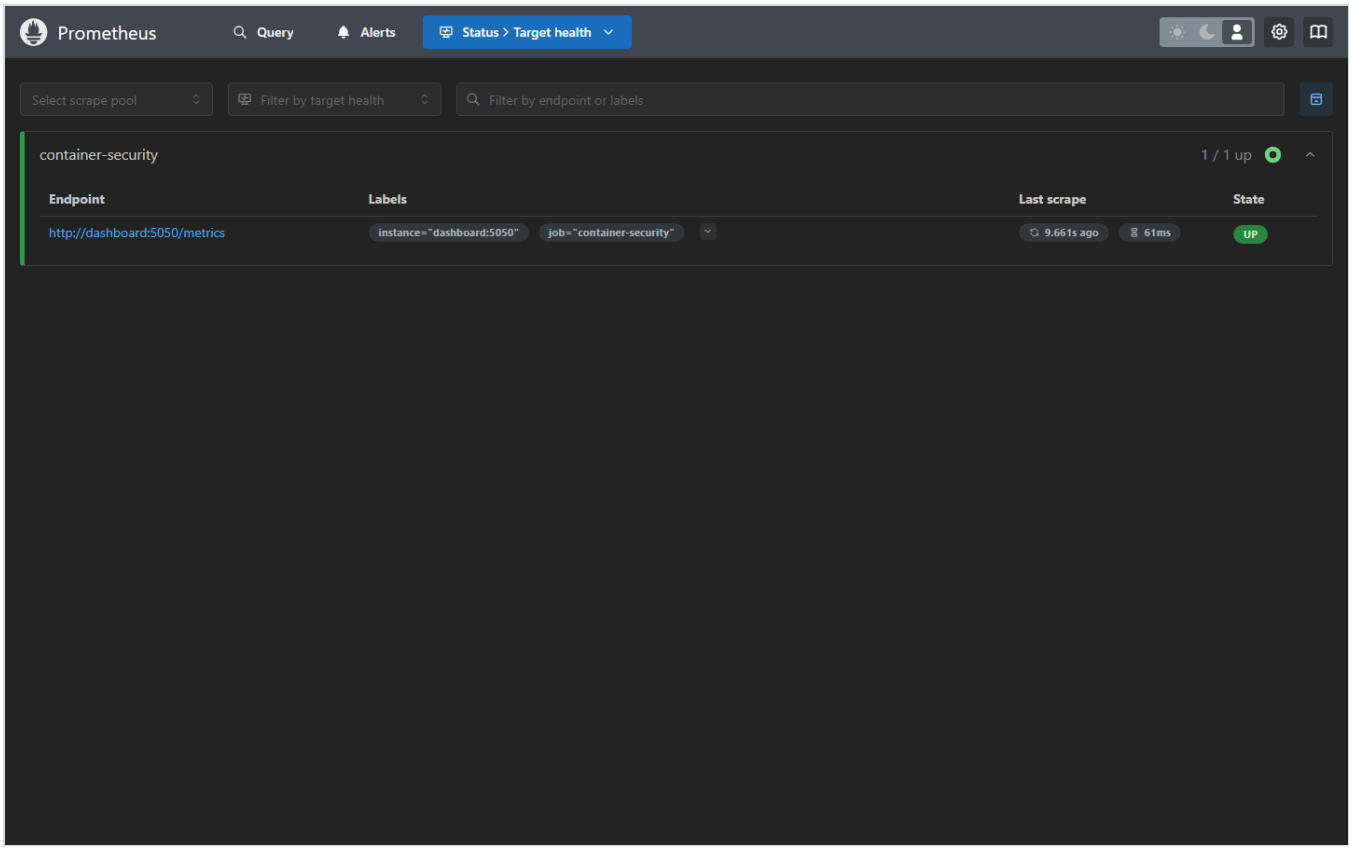



Figure 9. Genuine Prometheus target-health capture.

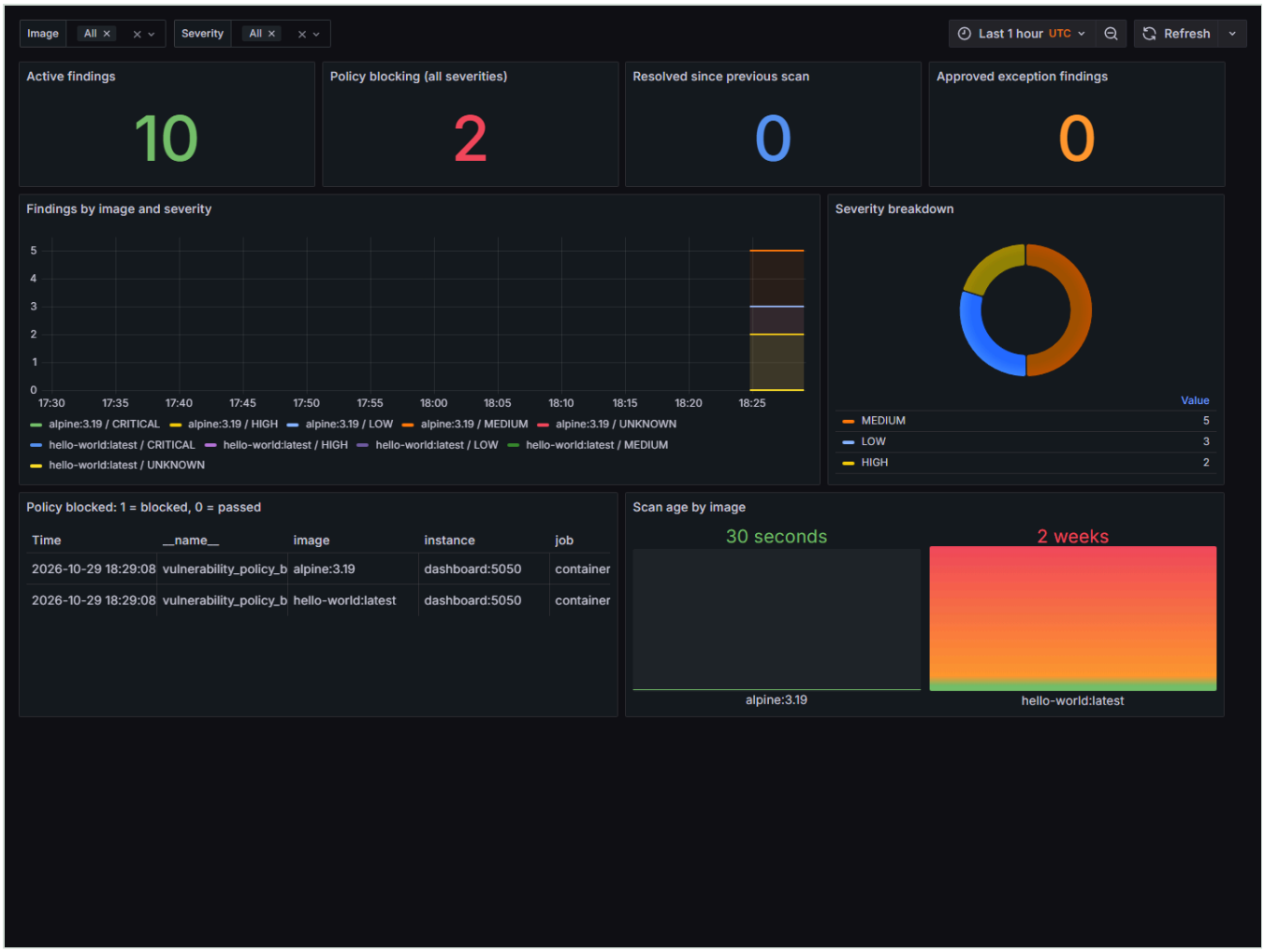



Figure 10. Genuine Grafana monitoring capture. Prometheus history begins with scraping; old SQLite timestamps are not backfilled as artificial samples.

Browser verification is archived in the original browser-check JSON. Checks cover 1440x1080 and 390x844, API/UI totals, filters, empty and invalid-date states, scan drilldown, deep-linked exception audit, nonblank canvas pixels and no JavaScript errors.

## 11. Sprint 5: Governed Exceptions, Rescans and Reliability

Exception rules require an exact advisory ID, a nonempty reason and a real YYYY-MM-DD expiry. Optional image/package scopes also match exactly; wildcards and duplicate scopes are rejected. The default production allowlist is empty. Expiry is inclusive through the specified UTC date, evaluated at scan time. Missing or malformed configured policies fail before scanning.

Suppression occurs before gate evaluation while preserving raw counts, approved findings, scope, justification, expiry and policy checksum. Implicit Trivy ignorefiles are disabled to prevent unaudited suppression. Accepted risk is distinct from a vulnerability being fixed.

| Real 29 October Alpine demonstration | Active / suppressed | Exit | Original evidence |
|---|---|---|---|
| Temporary non-production approval | 8 active; 2 HIGH suppressed; 0 blocking | 0 | approved.json |
| Expired rule | 10 active; 0 suppressed; 2 HIGH blocking | 1 | expired.json |
| Missing configured exception file | No scan performed | 2 | Deterministic tests and Sprint 5 execution record |

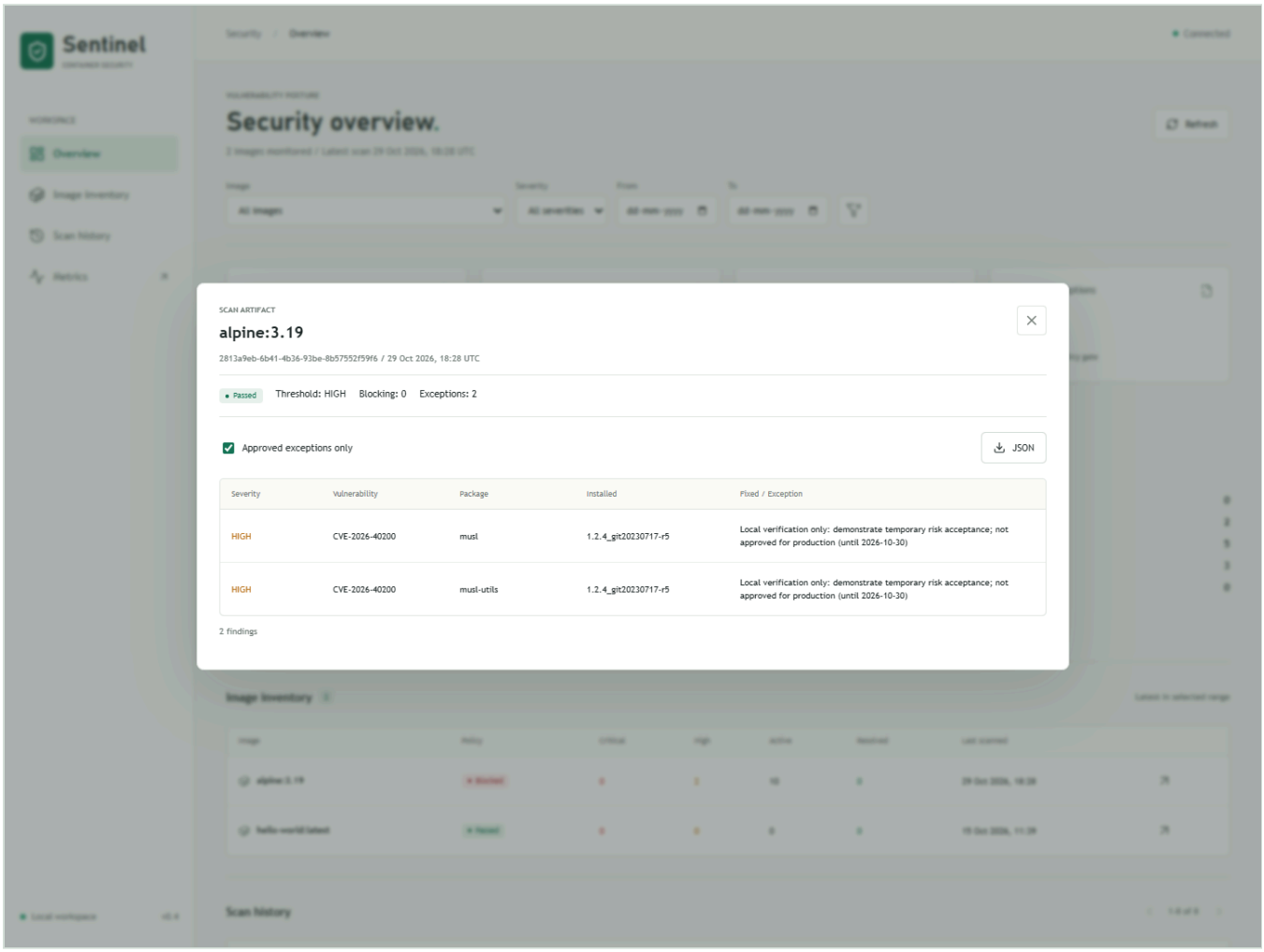



Figure 11. Genuine exception audit capture. The dialog describes the approved historical observation; a later blocked inventory may remain visible behind it.

## Fresh Rescan Lineage

Baseline `fa356ec8-553d-4769-bb81-20aeb03b599b` at `2026-10-29T13:57:51.161781Z` was followed by a real new Trivy scan `3531bf6a-0b7e-491d-aebb-b4d881adb621` at `2026-10-29T13:57:53.922734Z`. Its `rescan_of` points to the baseline. Both were blocked; no remediation was invented. The artifacts are linked in the evidence register.

### Reliability Controls

Defaults permit two retries after the initial attempt with 1s/2s exponential delays. Recognized transient network/registry conditions and process timeouts may retry; authentication failures, missing manifests/executables and invalid JSON fail fast. Each attempt has the configured Trivy timeout plus a 30s outer deadline. Database refresh shares the classified retry helper. Atomic replacement prevents partial successful-looking JSON artifacts.

Recovery, exhaustion, permanent failures, deadlines, malformed result structures and write failures are covered by controlled deterministic tests. A live registry outage or network fault injection was not performed and is not claimed.

## 12. Verification Matrix

| Verification | Evidence / observed result | Status |
|---|---|---|
| Integrated tests | 77 deterministic tests; archived JUnit: 0 failures, 0 errors | **Verified** |
| Current final-report verification run | Fresh JUnit and original test output | **Evidence attached** |
| Current main CI | Run #31 / 399d1af; success; scan/tests/reports/upload completed | **Verified** |
| Clean and vulnerable scan paths | Original final-version core log/artifacts plus historical real CI pass/block captures | **Verified** |
| Report integrity | Original JSON/HTML/PDF manifests match archived bytes, including clean Git checkout verification | **Verified** |
| Slack transport | Jenkins HIGH/CRITICAL and daily/weekly delivery logs | **Verified on Jenkins** |
| GitHub Slack | Repository secret absent; notification step skipped | Owner setup pending |
| Historical UI and monitoring | Real screenshots, browser-check JSON and observed exporter/Grafana health | **Verified at capture** |
| Approved and expired exceptions | Original active/suppressed artifacts and gate exits 0/1 | **Verified** |
| Rescan lineage | New UUID/time and rescan_of; actual new scan invocation | **Verified** |

Milestone test counts changed as scope grew: Sprint 2 had 16 tests, Sprint 3 had 28, isolated Sprint 4 had 35 and the integrated final suite has 77. These are separate historical states, not contradictory totals. Local captured browser history currently contains eight genuine observations, not a claim of large-scale production benchmarking.

# 13. Security, Limitations and Outstanding Actions

- Webhooks belong only in secret stores. The previously exposed webhook must be rotated by the account owner; its value is not reproduced here.
- GitHub and Jenkins secret stores are separate. GitHub Slack delivery remains unverified until its repository secret is entered and delivery is rerun.
- Channel-side Slack screenshots remain unavailable. A delivery-success log proves transport acceptance, not independent visual receipt.
- Allowlist changes need human review and branch controls. The code does not establish an approver identity or enforce organization governance by itself.
- Loopback binding, read-only API, nonroot dashboard container and restrictive web headers reduce exposure. Flask development hosting and anonymous Grafana Viewer are not a public production security model.
- Monitoring history requires explicit artifact download/import. No central CI collector, remote write API, Node Exporter or Pushgateway is deployed.
- Prometheus rules cover stale scans/exporter health/critical findings. Alertmanager-based external delivery is not configured.
- Registry/image tags and advisory databases change. Pinned scanner versions alone do not make future scan counts deterministic.
- Absence of a finding within the selected scan scope is not proof of patch installation or absence of all security risk.
- Python dependencies are bounded ranges; action references use release tags rather than universal commit-SHA pinning. Broader supply-chain hardening is future work.
- No paid infrastructure is provisioned. Local compute, downloads, service quotas, retention and academic Jenkins availability still have operational constraints.

# 14. Reproduction and Handover

From the cloned repository root, install Python dependencies and Trivy 0.74.0 on PATH. Registry-reference scans do not require the Docker socket. Docker Desktop/Engine is needed for the optional monitoring stack.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
trivy --version
python -m pytest tests -q

python -m scanner.trivy_scanner hello-world:latest
# An Alpine HIGH-threshold scan may intentionally return exit 1.
python -m scanner.trivy_scanner alpine:3.19 --fail-on HIGH
$artifacts = @(Get-ChildItem scan-results -Filter '*-????????.json' | ForEach-Object FullName)
python -m reporters.cli @artifacts --formats json,html,pdf --output-dir reports
python -m dashboard.cli @artifacts --metrics-file reports/scan-metrics.prom
python -m dashboard.app --port 5050
```

Open the local dashboard at http://127.0.0.1:5050. To run the provisioned exporter/Prometheus/Grafana stack, set a private `GRAFANA_ADMIN_PASSWORD` directly in the terminal and run `docker compose -f`

`dashboard/compose.yaml up -d --build`. Never paste the password or webhook into documentation or chat. Do not run `down -v` unless deleting monitoring history is intended.

CI evidence can be downloaded and imported using the same CLI. Rescan an original normalized artifact with `python -m scanner.trivy_scanner --rescan-artifact previous.json --refresh-db --output-dir scan-results/rescan`. Preserve the previous file and import both observations for historical comparisons. Full operational detail is in the README.

## 15. Challenges and Architectural Decisions

| Challenge | Decision / outcome |
|---|---|
| Policy failure confused with tool failure | Separate exits distinguish accepted execution with findings from configuration/operational errors. |
| Evidence lost when a gate stops CI | Preserve the scan result through reporting and attempt archive/upload before final outcome restoration. |
| Ephemeral CI cannot be a persistent metrics endpoint | Use a separate local persisted history exporter rather than pretending a stopped CLI remains scrapeable. |
| Count-only remediation metrics hide finding churn | Compare exact advisory/package sets, including suppressed findings. |
| Risk approval could silently hide findings | Mandatory reason/expiry, exact scope and separate suppression audit; no default approvals. |
| Different CI agent environments | Use shared Python logic, pinned Trivy and bootstrap pipelines; keep provider-specific orchestration separate. |
| Original diagram mixed roadmap and implementation | Replace it with deployment boundaries, persisted stores and labelled interfaces; retain old image as historical overview. |
| Evidence presentation could be mistaken for execution | Exclude reconstructed terminal panels from primary proof and provide original files, real captures and provenance. |

## 16. Evidence Register and Provenance

Core command-output logClean artifact / Vulnerable artifactMain run metadata / jobs / artifactsGitHub passing capture / blocked captureJenkins passing capture / blocked consoleSprint 3 delivery recordsGenerated report manifestBrowser checksDesktop / mobilePrometheus / GrafanaApproved / expired / audit captureBaseline / rescanMilestone JUnit / final-verification JUnit / test outputDeployment architecture

| ID | Evidence | Type and interpretation |
|---|---|---|
| E01 | Fresh actual command output; not a rendered terminal panel | |

| ID | Evidence | Type and interpretation |
|---|---|---|
| E02 | Original normalized scanner outputs from final verification | |
| E03 | Original public GitHub API responses for run #31; live URL remains authoritative for later changes/retention | |
| E04 | Genuine historical browser screenshots | |
| E05 | Genuine historical browser screenshots | |
| E06 | Verified Jenkins transport logs and execution details; channel screenshots not available | |
| E07 | Original JSON/HTML/PDF bundle with per-file checksum and size | |
| E08 | Captured checks, timestamps and observed totals | |
| E09 | Genuine local application captures | |
| E10 | Genuine running monitoring captures | |
| E11 | Original date-specific accepted/expired risk demonstrations | |
| E12 | Original observations with fresh UUID/time and linked lineage | |
| E13 | Machine-readable deterministic test evidence; distinct execution times | |
| E14 | Technical documentation, not an execution screenshot | |

Relative links resolve inside the repository after cloning. HTML files are rendered locally; GitHub normally displays their source. Jenkins URLs may require academic credentials. Report PDFs embed the figures, while original machine-readable evidence remains in the adjacent repository paths.

# 17. Milestone Appendices

1. Sprint 1: Setup and Basic Scanning - historical design and setup narrative; reconstructed panels are not independent proof.
2. Sprint 2: CI/CD Integration - milestone 16-test and pass/block evidence.

3. Sprint 3: Reporting and Notifications - original report/transport/default-cadence evidence and its later updates.
4. Sprint 4: Dashboard and Monitoring - isolated 35-test milestone and genuine browser captures.
5. Sprint 5: Exceptions and Reliability - integrated 77-test evidence, original audit artifacts and limits.
6. Viva Preparation - 93 searchable questions aligned to the implementation.

Appendices preserve their original dates, version descriptions and evidence context. This consolidated report supersedes obsolete planned-versus-delivered statements for the final Sprint 1-5 state without rewriting historical screenshots.

## 18. Conclusion

The delivered system covers the requested path from first image scan to repeatable CI enforcement, consistent report evidence, real Jenkins notification delivery, persistent history, monitoring and governed vulnerability exceptions. Shared code paths, explicit failure semantics and retained original artifacts make the result inspectable rather than dependent on screenshots alone.

The implementation and evidence support completion of Sprints 1-5. Production deployment, remaining account-secret/rotation tasks and channel-side visual proof remain explicit boundaries. No reconstructed panel, skipped Slack step, manually triggered schedule or accepted exception is represented as something it does not prove.

---

Tabrez Ajaz | Container Image Vulnerability Scanner with Reporting | Final consolidated report, 30 October 2026 | Source baseline 399d1af